

These measurements help determine whether the cache is reducing LLM usage effectively and whether further optimisation is required.

Tracing cache behaviour is equally important. Reviewing cache traces can reveal whether the similarity threshold should be adjusted or whether new categories of user queries have emerged that are suitable for caching.

Monitoring platforms can also provide accuracy metrics, including true positive and true negative rates, alongside usage statistics such as memory consumption and token savings. Together, these measurements help maintain both retrieval quality and application performance over time.

Implementing semantic caching

Several implementation approaches are available depending on an application's architecture and the level of management required.

Developers can build a custom implementation using RedisVL, an open source library that provides semantic caching together with additional AI capabilities. Developers can configure cache entries, update responses, define similarity thresholds, manage expiration policies and customise cache behaviour according to application requirements.

RedisVL also supports embedding caching. Since generating vector embeddings requires computational resources, repeatedly calculating embeddings for identical queries introduces unnecessary overhead. By caching embeddings alongside responses, applications can reduce both compute requirements and response times.

Applications built with LangChain can integrate semantic caching through the Redis middleware, which supports both semantic response caching and tool caching. Tool caching stores the results of previous tool invocations, preventing repeated tool selection and execution when identical requests are received.

Contributing to open source

Open source projects provide an effective way to gain practical experience with emerging technologies. Redis maintains a collection of open issues within its RedisVL project, offering contributors an opportunity to work on semantic caching and other AI-related features.

For developers interested in learning semantic search, vector databases and AI infrastructure, contributing to these projects provides valuable hands-on experience while helping improve the wider open source ecosystem.

For organisations seeking a fully managed solution, LangCache provides semantic caching through a simple API. Embedding generation, cache management, scaling and availability are handled automatically, allowing developers to focus on application development rather than infrastructure management.

Practical considerations for production deployments

There are several practical considerations when deploying semantic caching in production systems.

One concern is ensuring that cached responses do not replace workflows that require retrieval-augmented generation (RAG). A common implementation pattern is to check the semantic cache first. If a matching query already exists, the cached response is returned immediately. Otherwise, the application proceeds through the complete RAG pipeline, generates a fresh response using the LLM and stores the new result in the semantic cache for future requests.

Another important consideration is hallucination. Because semantic caching retrieves previously generated responses from a vector database rather than generating new text through an LLM, responses remain deterministic. The semantic router performs intent classification using vector search instead of language model inference, further reducing variability.

Many production AI systems minimise unnecessary LLM

usage wherever possible by using deterministic components such as vector databases for routing, retrieval and intent classification.

Managing stale information is another important aspect of deployment. Information that changes frequently should not remain in the cache indefinitely. Appropriate TTL values and eviction policies ensure that outdated responses are removed and replaced as new information becomes available.

Finally, semantic caching is generally less suitable for multi-turn conversations. As conversations progress, later questions often depend on the context established by previous exchanges. Since semantic caching primarily compares individual queries, policy documents, FAQs, and other relatively stable knowledge sources are typically better candidates for caching than long conversational sessions.

When combined with vector databases, appropriate embedding models, carefully selected similarity thresholds and continuous monitoring, semantic caching can substantially reduce token usage while improving response times. Successful adoption, however, depends on selecting suitable use cases, designing effective cache policies, and continuously evaluating application behaviour. Rather than replacing LLMs, semantic caching complements them by ensuring that language models are used only when genuinely necessary. **END** 🐙

This article is based on the talk 'Reduce LLM Costs with Semantic Caching: Real World Lessons' by Bhavana Anant Giri, developer advocate, Redis, at AI DevCon India 2026. It has been transcribed by Ananthu Ashok of the EFY Group.